

Sharpa Wave - Vision Based Teleoperation

USER MANUAL

Contents

1. Overview

1.1 Functionalities3

1.2 Requirements3

2. Setup

2.1 Hardware4

2.2 Software7

2.3 Calibration Guide7

3. Developer Notes

3.1 Algorithm Explanation8

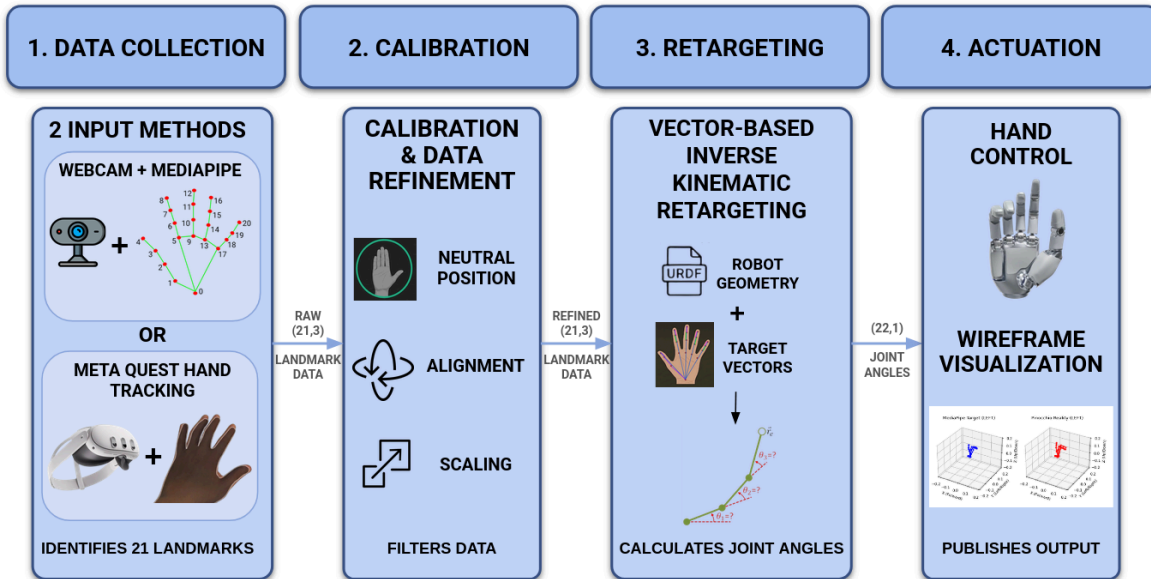
3.2 Key Files10

3.3 YAML Walkthrough10

3.4 Notes for Sharpa Wave13

Overview

SHARPA WAVE VISION BASED TELEOPERATION



This system is a ready-to-use vision-based teleoperation platform for the Sharpa Wave hand. It supports mono or bimanual control via any standard RGB camera, including a regular laptop webcam, as well as the Meta Quest VR headset. The software includes 3 main components: landmark detection, data refinement based on calibration data, and vector-based retargeting. The following documentation includes instructions for installation, set up, and teleoperation, as well as a description of the pipeline, highlighting key concepts and relevant files.

1.1 Functionalities:

Both teleoperation modes, either camera based or VR headset, follow the following structure. At the start of teleoperation, there is a 3 second calibration stage. It is recommended to recalibrate before every session or user. After the calibration stage, the user may select the desired functionalities, which include real-time control as well as an additional, optional plotted visualization of the gathered and retargeted data. To end the teleoperation at any point, press 'q' while in the opened window. If no hand is not detected within the frame while in control mode, the Sharpa Wave hand will automatically return to the zero position.

1.2 Requirements/Dependencies:

- Sharpa Wave Hand and SDK
- Recording Device (embedded webcam, external camera, Meta Quest, etc)
- 64-bit Linux Operating System

Set Up

2.1 Hardware:

The main hardware components are the Sharpa Wave hand and the data collection device. Firstly, connect the Sharpa Wave hand(s) and verify that it is connected via launching Sharpa Pilot or pinging the devices directly.

Dual-Hand Configuration

Two network setups are supported for dual-hand configuration: using a network switch (recommended) or using multiple Ethernet interfaces.

1. Using a network switch (recommended):

- Connect the host and both devices via a switch.
- Use a single host IP (e.g., [192.168.10.240](#)) as the tactile destination IP.
- Ensure all devices are on the same network segment.

2. Using multiple Ethernet interfaces:

- Each host interface must have a unique IP address.
- Each hand and the corresponding host interface must be on the same subnet.
- Each hand must be configured with a tactile destination IP matching the corresponding host interface.

When trying to connect two hands for bimanual control, there can be complications with overlapping Ethernet threads that prevent a proper connection.

If starting on a new device, the Ethernet may not appear at all initially as the hardware may not be loaded. If the driver has been successfully loaded, the following command should print the name of the ethernet driver's hardware to the terminal. If not, the appropriate driver will need to be installed. On Linux, the command to check the physical driver attached for Ethernet if connecting through PCI Ethernet is

Shell

```
lspci | grep -i ethernet
```

The next issue, especially when connecting multiple hands, is overlapping IP addresses, especially for the first connection. In order for a successful connection, the computer and each hand should each have unique IP addresses. For the left hand, the appropriate IP address is 192.168.10.10, and for the right hand, the appropriate IP address is 192.168.10.20. However, the computer may try to use a similar IP address, such as 192.168.10.10/0. This can be investigated by checking what IP address is being used by which connections. In order to see what devices are being identified and connected, run

Shell

```
nmcli connection show
```

This will provide the NetworkManager's data on the ethernet devices. On a Linux system, the following naming conventions are common. If using the PCI Ethernet port, the computer IP address should be identifiable by the "*enp8s0*" interface. If using the USB Ethernet port, the computer's IP address should be identifiable by the "*enx....*" interface. Any future commands will be based on trying to connect a right Sharpa Wave hand through the standard PCI Ethernet port. Thus, *enp8s0* will be used as the reference for the interface name. A left Sharpa Wave hand will be connected to the USB ethernet port and will use the *enx00e04c6d55b5* interface name. The corresponding names of the devices assigned by the NetworkManager should be visible as well. The following two commands should be run for each port that is not functioning properly:

Shell

```
ip -4 addr show enp8s0
ip neigh show dev enp8s0
```

The first command will specify the IP addresses being utilized by each port, and the second command will return the neighbor table, which should say REACHABLE if it is able to connect to another device on that address. If the IP address overlaps with the hand, such as "192.168.10.10/0", or the neighbor table is empty, the conflicting addresses are preventing connection. It is possible that the laptop will try to connect both IP addresses of the hand to a single Ethernet port. In this case, it is essential to make sure that *enp8s0* and *enx00e04c6d55b5* route to the corresponding IP addresses of the hands that they are connected to. If one port is trying to connect to both, run

Shell

```
sudo ip route replace 192.168.10.20 dev enp8s0
sudo ip neigh del 192.168.10.20 dev enx00e04c6d55b5
```

to adjust the ports accordingly. In this code, the right Sharpa Wave hand is connecting only to the PCI port and being deleted from the USB ethernet connection.

Another possible room for error is the Ethernet driver not functioning properly. This is especially possible if the Ethernet port is not showing up despite the hardware being installed. Once the appropriate driver has been identified and is free for use, load the driver. The following command uses r8169 as the default driver.

```
Shell
```

```
sudo modprobe r8169
```

After this command, the driver should start recognizing the Ethernet port and connect properly. This can be verified by running the following and ensuring the appropriate ethernet port is displayed.

```
Shell
```

```
nmcli connection show
```

Landmark Collection Device Configuration

If operating via a camera, the only configuration needed is to specify the camera path. By default, the code uses `/dev/video0`, which will operate the default camera, typically the in-built webcam. If connecting an external camera, specify the path as one of the arguments when running the program or adjust the `DEFAULT_CAMERA_PATH` variable at the top of *sharpa_wave_webcam.py*. Hand tracking via a camera works best with the palm facing the camera parallel to the screen.

If operating via the Meta Quest VR Headset, the following steps will need to be taken at installation. While Meta Quest does have a native SDK for hand tracking, it is primarily structured for game engines and thus requires a large overhead. Thus, the included file for Quest control utilizes a common third party app, Hand Tracking Streamer to transmit raw spatial data via Wi-Fi. Thus, the control computer and Quest must be on the same network. The app can be downloaded from the Meta Quest store, and the SDK can be installed via

```
Shell
```

```
pip install hand-tracking-sdk
```

Once installed, the app will prompt for the path to transmit the data. Select UDP for the method for low-latency packets. Both devices, the Meta Quest and the controlling computer, need to be connected to the same WiFi network for the data transmission to occur. Enter the computer's IP address for the stream address, which can be found by running the following command, with the appropriate WiFi driver mentioned (it will likely start with `wlp` and can be identified when running `nmcli connection show` when connected to WiFi). The following code uses `wlp14s0` as the driver.

```
Shell
```

```
ip -4 addr show wlp14s0
```

The port can be configured with the `DEFAULT HTS_PORT` variable at the top of the `sharpa_wave_quest.py` file, but it is recommended to leave it as standard at 9000. Choose whether one or multiple hands are transmitted and then select start streaming before running the retargeting program.

Hand tracking via VR goggles will work similarly whether the palm is facing toward or away from the camera. However, the orientation of the hand will affect the obstructions faced by the camera. Pinching with the thumb and index finger is often better captured with the palm facing away, while pinching with the ring and pinky fingers are often best captured with the palm facing toward the camera. In order to stop streaming and return to the menu, pinch the thumb and index finger with the left hand when the palm is facing toward the camera.

2.2 Software

Install commands

Reference `README.md` for instructions on installation, repo layout, and dependencies for installation.

File Naming Conventions and Terminal Commands:

When calling `sharpa_wave_webcam.py` or `sharpa_wave_quest.py`, it takes up to 3 arguments:

- config-path (The path to the .yaml configuration file(s))
- robot-dir (The path to the directory which stores the .urdf files)
- camera-path (The path to the webcam (/dev/video0 on most Linux systems))

You can also input these paths directly as the default settings by adjusting the `DEFAULT_PATH` variables at the top of the `sharpa_wave_webcam.py` or `sharpa_wave_quest.py` file.

2.3 Calibration Guide

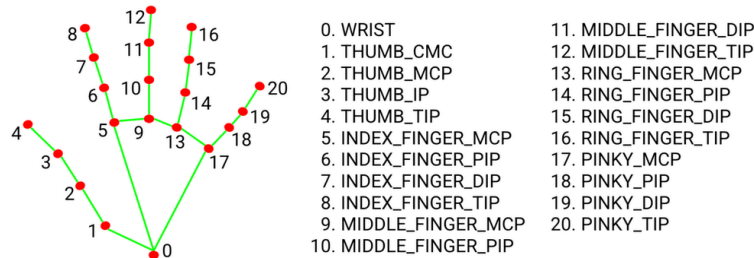
The calibration stage at the beginning of teleoperation creates a reference for the teleoperator's hand, including sizing and neutral position. During this stage, the hand is expected to be held in an upright, neutral position, with the fingers extended straight vertically and the palm oriented in the default position during the control as shown below. The thumb may be slightly flexed. Maintaining this neutral position during teleoperation is recommended for optimal performance.



Developer Notes

3.1 Algorithm Explanation

The software allows for real time mono or bimanual control of the Sharpa Wave hand. The first step is landmark detection. For camera-based teleoperation, Google's MediaPipe, a model that can detect a hand within an image, to process the camera feed is used. MediaPipe detects 21 landmarks on the hand along the main joints, as documented below.



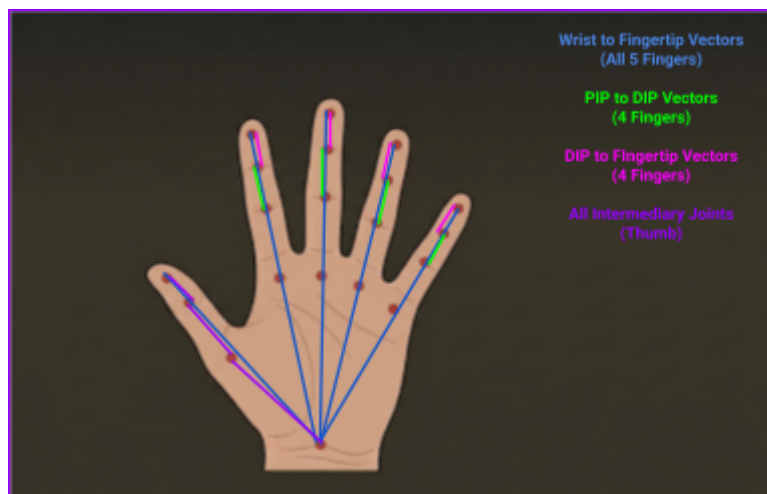
This skeleton is mapped onto the webcam in real time. MediaPipe naturally extrapolates 3D positions for each landmark from the monocular RGB image based on its comprehensive anatomical training data and any depth clues it sees in the image, such as foreshortening or scale changes from the camera. However, to compensate for any errors in the projection, the calibration stage captures a neutral position as a reference frame. This helps filter through any noise or inaccuracies due to the MediaPipe recreation of the 3D projection. Most of the relevant MediaPipe code is within the *single_hand_detector.py* file, which identifies the landmarks of the hand and performs some initial matrix calculations to map to the correct anatomy based on the hand and adjust for the warping in the frame.

For Meta Quest control, the Hand Tracking Streamer app transmits the same 21 landmarks along the joints via UDP packets. Due to the multiple RGB and IR cameras, the model is able to put together a more accurate perception of depth as compared to the MediaPipe data. However, the Meta Quest is more susceptible to errors due to occlusion, especially when bending the ring and pinky fingers behind the palm. The relevant return from this portion of the algorithm is an array sized (21, 3), holding the x, y, z coordinates for each of the 21 landmarks.

This raw data is then refined based on the average neutral position data collected during the calibration process. For all frames, to account for any misalignment with the camera, the calibration recalculates the XYZ coordinate axes according to landmarks on the palm. The X axis is rotated to correspond to the vector along the knuckle line, the Y axis along the fingers, and the Z axis along the palm normal. During the calibration stage, an ideal template hand is constructed that follows the neutral pose meant to be maintained during calibration. To account for any skews during the neutral position and ensure the hand remains upright, the calibration algorithm adjusts the forward tilt of the MCP joint, or the base knuckle closest to the palm, in the four fingers (excluding the thumb). This joint is commonly affected due to misperceptions in depth, especially from MediaPipe data. It also straightens the four fingers by projecting the

middle PIP and DIP joints (the second and third knuckles) along a straight line from the MCP joint to the fingertip and adjusting the landmarks accordingly. Based on this ideal, straightened hand in neutral position, the calibration calculates an 'offset' value, which it applies to every frame within the live teleoperation. If a custom scaling factor is not included, it additionally scales the data based on the length of each finger and the size of the palm to accurately match the robot URDF. After the calibration stage, the user can choose whether they want to view the plots, control the robot hand, or both. The calibration data for each hand is logged in a unique .JSON file, and the path can be specified in the DEFAULT_CALIBRATION_PATH variable.

After appropriately adjusting the (21,3) array of landmark locations, the array is fed to the DexRetargeting algorithm, located in the *optimizer.py* file. This algorithm references the physical geometry of the robot through its URDF file and calculates the appropriate joint angles (in radians) of the hand based on its optimization algorithm. The algorithm calculates the vectors from a list of origin or reference points to a list of target points, and then minimizes the difference between the MediaPipe vector and the robot reconstruction. Thus, it takes an input of (N, 3) array, where N represents the number of joints of interest, and returns a (22,1) array for each joint angle in the Sharpa Wave hand. While only wrist to fingertip vectors are required for basic functionality, the default recommended structure optimizes those vectors for each of the four main fingers as well as matching the vectors between the PIP and DIP joint as well as the DIP to fingertip. This is due to the many degrees of freedom allowing for multiple possible orientations of the hand to match the fingertip positions, many of which do not accurately reflect the whole behavior. Because the MCP creates the largest pivot arm, it is often overutilized by the optimization algorithm, leading to straight fingers that bend from the base knuckle. This is often a poor pose for grasping or pinching, and thus, the intermediary points along the upper half of the hand have been included. Due to the discrepancies between a human thumb and the Sharpa Wave thumb, each intermediary vector along the joints is being optimized to ensure the most accurate reconstruction of the hand pose. Thus, N is defaultly configured to 16. The following diagram illustrates the chosen vectors.



3.2 Key Files:

The files *sharpa_wave_webcam.py* or *sharpa_wave_quest.py* (depending on the control mode) control the overall pipeline and have been configured for the Sharpa Wave hand. The rest of the files mentioned can be found within the `supplements` directory. The code has three main threads for parallel processing. Firstly, the webcam data is captured, and the data for each hand is passed to the *teleop_hand.py* file, where each hand is processed independently. The data is refined and passed to the retargeting solver on unique threads to reduce latency. The threads are merged before being pushed to the necessary processing files (visualization or robot control) to remain synchronized throughout frames.

The calibration algorithm is found within the *hand_calibration.py* file, which initializes a 3 second calibration stage for each hand before the tele-op control commences. This file includes the real-time adjustments that are made to the data for refinement as specified in the algorithm explanation section above. After the calibration stage, the user can choose whether they want to view the plots, control the robot hand, or both.

The real-time plotting is controlled in the *hand_visualization.py* file, which plots both the captured data as well as a reconstruction of the robot's position calculated from the optimization algorithm. This reconstruction is done through Pinocchio, which creates a model of the robot based on the dimensions in the URDF and calculates the kinematics of the landmarks based on the angles provided. This file allows for direct comparison of the refined data as well as the angles calculated by the retargeting algorithm and is thus very useful for debugging performance.

3.3 YAML Walkthrough

The algorithm relies on a YAML file to specify the parameters for the optimization, which can be adjusted as seen fit based on the desired application. An example `.yaml` file is included below. The naming convention of the YAML files is used to identify the number and type of hands being controlled. The YAML file must have the type of hand (either "right" or "left") within the file name. The program also requires the URDF files of the robot. This has been included within the repo, but the directory can also be downloaded from the Sharpa [GitHub](#).

```

1 retargeting:
2   type: vector
3   urdf_path: left_sharpa_wave_with_wrist.urdf
4
5
6   target_origin_link_names: [
7     "left_hand_C_MC",
8     "left_hand_C_MC",
9     "left_hand_C_MC",
10    "left_hand_C_MC",
11    "left_hand_C_MC",
12    "left_hand_C_MC",
13    "left_thumb_MC",
14    "left_index_PP",
15    "left_middle_PP",
16    "left_ring_PP",
17    "left_pinky_PP",
18    "left_thumb_DP",
19    "left_index_DP",
20    "left_middle_DP",
21    "left_ring_DP",
22    "left_pinky_DP",
23  ]
24
25
26   target_task_link_names: [
27     "left_thumb_fingertip",
28     "left_index_fingertip",
29     "left_middle_fingertip",
30     "left_ring_fingertip",
31     "left_pinky_fingertip",
32     "left_thumb_MC",
33     "left_thumb_DP",
34     "left_index_DP",
35     "left_middle_DP",
36     "left_ring_DP",
37     "left_pinky_DP",
38     "left_thumb_fingertip",
39     "left_index_fingertip",
40     "left_middle_fingertip",
41     "left_ring_fingertip",
42     "left_pinky_fingertip",
43  ]
44
45
46   scaling_factor: 1.0
47
48   target_link_human_indices: [
49     [0, 0, 0, 0, 0, 0, 2, 6, 10, 14, 18, 3, 7, 11, 15, 19],
50     [4, 8, 12, 16, 20, 2, 3, 7, 11, 15, 19, 4, 8, 12, 16, 20]
51  ]
52
53   low_pass_alpha: 0.4
54

```

The `type` field specifies between vector or position retargeting. The vector setting optimizes the similarity between the human and robot hand according to the specified vectors in the YAML file. Position based retargeting, in contrast, seeks to directly match the xyz coordinates. Vector-based retargeting is *highly* preferred as it better optimizes the overall hand shape and is less susceptible to translation or movement of the hand.

The `urdf_path` contains the path relative to the `robot_dir` argument specified earlier, which stores the URDF files. The code is configured to default to the paths for the files included with this package, but can be adjusted if the URDF files have been previously downloaded. There are URDF files for both with and without the wrist, depending on how the hand is mounted.

The `target_origin_link_names` list should contain the names of all of the starting points for the vectors of interest, and the `target_task_link_names` should contain the names of the end point of the considered vectors. These names can be found after the “link” tag within the URDF as highlighted for the wrist joint (left_hand_C_MC) in the screenshot below. These are the parameters that are best to adjust and tune based on the desired tasks. At the bare minimum, the wrist to fingertip vectors are essential.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <robot name="left_sharpa_wave">
3   <mujoco>
4     <compiler meshdir="meshes" balanceinertia="true" discardvisual="false" />
5   </mujoco>
6   <link name="left_hand_C_MC">
7     <inertial>
8       <origin
9         xyz="-0.00216 -0.00536 0.04011"
10        rpy="0 0 0"
11      />
12      <mass value="0.6506" />
13      <inertia
14        Ixx="0.000674245"
15        Ixy="-2.498E-06"
16        Ixz="-7.348E-06"
17        Iyy="0.000793354"
18        Iyz="1.05749E-04"
19        Izz="0.000294753"
20      />
21    </inertial>
22    <visual>
23      <origin
24        xyz="0 0 0"
25        rpy="0 0 0"
26      />
27      <geometry>
28        <mesh filename="package://left_sharpa_wave/meshes/left_hand_C_MC_visual.STL" />
29      </geometry>
30      <material name="">
31        <color rgba="0.79216 0.81961 0.93333 1" />
32      </material>
33    </visual>
34    <collision>
35      <origin

```

The `scaling_factor` is meant to account for differences in the user’s hand size and the robot hand. However, due to the variations in proportions from the robot to a human hand as well as the variations between trials, using a single universal `scaling_factor` is not recommended for optimal performance. The code includes a custom scaling calculation for the palm and each finger individually from the data collected through the calibration stage. Thus, the parameter should remain 1.0 to stay compatible with the format expected by the dex-retargeting optimizer

while preventing any duplicate scaling. However, if an alternative scaling factor is desired, one can adjust this parameter to any non 1.0 value, and the code will bypass the custom-scaling functionalities and apply the desired scaling factor.

The *target_link_human_indices* matches the specified joints to their corresponding indices in the landmarks array. The first list is the indices of the origin link names, and the second list is the indices of the target link names. It is imperative that the indices across these lists should match the ordering in the *target_origin_link_names* and the *target_task_link_names* lists.

The *low_pass_alpha* is a filtering and smoothing parameter to dampen frame-to-frame movement when the position has not changed significantly. The lower the parameter, the more filtering will be applied. This prevents jittery motions, but it may increase the latency. The recommended default is 0.4.

3.4 Notes for Sharpa Wave

Joint Mapping

The first crucial element for repurposing the DexRetargeting algorithm is mapping the calculated joints to the appropriate positions. When sending a (22,1) array of joint angles to the Sharpa Wave, the *set_joint_position* function expects the angles to correspond to a prespecified order. Thus, a mapping array, *JOINT_NAMES*, is defined, linking the joint names from the URDF to the corresponding indices in the array. We can map the result of the retargeting algorithm, *qpos*, to have the appropriate angles at the correct position within the array before passing it to the hand.

Joint Space Calibration

The landmark data does not neatly correspond to the correct axes for the Sharpa Wave hand. Thus, prior to sending to the retargeting solver, the landmark data array is rotated, establishing the x, y, and z frames based on the vectors across the hand. This also improves the accuracy of the control even when the hand is not perfectly flat in comparison to the webcam, as the joint space is defined in reference to the palm's normal and parallel axes.

Latency/Control Frequency

The camera only stores the most recently captured frame, ensuring that only real-time data is processed, courtesy of the *frame_queue_utils.py* file. Notably, any motion between the calibration stage and selecting a control option is discarded. The capture rate of the webcam appears to be the main limiting factor in the control frequency of the system. A typical laptop webcam captures approximately 30fps, and thus the robot updates at 30Hz. For control via the Quest, the landmark detection is not limited by the frame rate of the webcam, and can thus be operated at up to 120Hz. The robot control and wireframe visualization run on different processes, and thus, when running with control and simultaneous visualization, the robot control maintains its optimal control frequency, while the wireframe updates every 5 frames to

prevent overloading the image loading system. When not controlling the robot hand, the visualization updates every frame for real time visualization.

Currently, for webcam-based teleoperation, the most computationally expensive and time consuming aspects of the algorithm is the landmark detection, which typically consumes up to 60% of the loop time. This can be improved by restricting the region of interest to narrow on the hand before processing all of the pixels, but this is primarily recommended for mono hand control. It can add unnecessary computation for bimanual control when the hands typically cover the majority of the visual frame. In order to improve the computational efficiency of the retargeting algorithm, which plays a significant part in the overall loop time especially for Quest control, one can relax the convergence parameter, which is the maximum distance between vectors allowed by the minimization algorithm before it stops calculating. The default is $1e-6$ for vector retargeting, but the control seemed comparably promising with $1e-5$ while providing an overall improvement of approximately 2ms per loop.

Limitations:

Due to the projection from 2D to 3D as well as the physical limitations of the Sharpa Wave's range of motion, tasks that require the hand to be curled extremely tightly against itself may be limited. This is especially true for the PIP joint, or the second knuckle, as bending only this joint places the rest of the finger directly in line with the camera, making depth interpolation difficult. Other occlusions tend to occur when pinching the thumb and index fingers. Because the key landmark tracks the center of the fingertip, in order to pinch or make contact between fingers, exaggerating the motion slightly may be recommended. Additionally, holding the hand slightly higher than the webcam and angling it downwards may improve performance as it prevents occlusion during pinching.